Depuración de programas 2
INF20125L

Felipe Dumont Ortiz
Felipe.dumont@usm.cl

Iniciemos lubuntu

- Utilizaremos la distribución **lubuntu** en los laboratorios
- Para ingresar en un laboratorio tienen que utilizar el usuario **.\test** y la contraseña **test**
- Abriremos dentro de esta sesión la máquina virtual de oracle, Oracle VirtualBox y **iniciaremos la máquina virtual** seleccionada lubuntu

Ejercicios

- Dentro de linux, descarguen los ejercicios subidos en Aula de laboratorio, con el nombre de “Depuración 3” y descomprímanlos.

Análisis de memoria
Valgrind

Valgrind nos Permite:

- Accesos inválidos a memoria
- Uso de memoria no inicializada
- Memory leaks
- Errores con malloc / free

Sin modificar nuestro código

¿Qué cambia para nosotros?

Igual que para utilizar GDB, tenemos que compilar nuestro programa utilizando el -g

Y luego llamando a Valgrind con nuestro programa compilado

```
gcc -g programa.c -o programa
valgrind ./programa
```

Valgrind

Hay que ver los resultados de valgrind y leerlos, normalmente si tenemos problemas puede que nos muestre cosas como:

- Invalid read/write
- Use of uninitialized value
- Definitely lost (memory leak)

```
gcc -g programa.c -o programa
valgrind ./programa
```

Resultados de Valgrind

```
==14785== Memcheck, a memory error detector
==14785== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==14785== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
```

```
==14785== Invalid write of size 4
==14785==    at 0x1091B7: llenar (valExtra2.c:11)
==14785==    by 0x10927A: main (valExtra2.c:35)
==14785==  Address 0x4a85054 is 0 bytes after a block of size 20 alloc'd
==14785==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==14785==    by 0x10926A: main (valExtra2.c:33)
```

```
==14785== HEAP SUMMARY:
==14785==     in use at exit: 20 bytes in 1 blocks
==14785==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==14785==
==14785== LEAK SUMMARY:
==14785==    definitely lost: 20 bytes in 1 blocks
==14785==    indirectly lost: 0 bytes in 0 blocks
==14785==      possibly lost: 0 bytes in 0 blocks
==14785==    still reachable: 0 bytes in 0 blocks
==14785==         suppressed: 0 bytes in 0 blocks
==14785== Rerun with --leak-check=full to see details of leaked memory
```

Ejercicio 1, 2 y 3

Continuando con los ejemplos de la semana pasada, revisen estos códigos pero ahora con valgrind y detecten los errores en estos programas.

Una vez que terminen todos los ejercicios sigan con los ejercicios que vienen.

Ejercicio 4

Ahora trabajaremos con asignación de memoria, detecte los problemas que existen en el siguiente programa utilizando valgrind y corrijalos, sabrá que funciona correctamente cuando valgrind no detecte problemas.

Ejercicio 5

Ahora tenemos asignación de memoria por lo que cuando ejecutemos valgrind, nos mencionará si liberamos correctamente las asignaciones de memoria.

Revise este caso y corrija el problema de memoria

Ejercicio 6

Detecte el problema que existe en este programa y corrijalo

Depuración de programas 2
INF20125L

Felipe Dumont Ortiz
Felipe.dumont@usm.cl

Ejercicios extra 1 y 2

Revise ambos programas que se encuentran en la carpeta EjercicioExtra, analícelos y corríjalos:

¿Qué deberían hacer los programas?
¿Hay algo que corregir ?
En caso de que sea necesario corregir, ¿Qué tengo que corregir?

Depuración de programas 2
INF20125L

Felipe Dumont Ortiz
Felipe.dumont@usm.cl